

LAB ASSIGNMENT – 5.5

NAME – SHREE PRIYA

2303A51465

BATCH – 29

Task Description #1 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

CODE:

```
def is_prime_naive(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    for i in range(2, n):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True
```

```
def is_prime_optimized(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    if n <= 3:
```

```
        return True
```

```
    if n % 2 == 0 or n % 3 == 0:
```

```
        return False
```

```
i = 5
while i * i <= n:
    if n % i == 0 or n % (i + 2) == 0:
        return False
    i += 6
return True

# Example usage
number = 29
print(f"Naive approach: Is {number} prime? {is_prime_naive(number)}")
print(f"Optimized approach: Is {number} prime? {is_prime_optimized(number)}")
```

```
C: > Users > Shree priya > Downloads > aac5-5.py > is_prime_optimized

1  #Naive Approach
2  def is_prime_naive(n):
3      if n <= 1:
4          return False
5      for i in range(2, n):
6          if n % i == 0:
7              return False
8      return True
9  #Optimized Approach
10 def is_prime_optimized(n):
11     if n <= 1:
12         return False
13     if n == 2:
14         return True
15     if n % 2 == 0:
16         return False
17     for i in range(3, int(n ** 0.5) + 1, 2):
18         if n % i == 0:
19             return False
20     return True
21 number = 29
22 print(f"Naive: Is {number} prime? {is_prime_naive(number)}")
23 print(f"Optimized: Is {number} prime? {is_prime_optimized(number)}")
24
25
26
```

PROBLEMS 2 OUTPUT TERMINAL PORTS

▼ TERMINAL

```
Optimized: Is 29 prime? True
The 6th Fibonacci number is: 8
Error: The file was not found. Please check the filename.
File processing attempt completed.
Enter username: avara
Enter password: 1286
Invalid username or password
Buddy says Woof!
Buddy is 3 years old.
Enter username: avara
Enter password: 1234
Invalid username or password
Buddy says Woof!
```

Task Description #2 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.

- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

CODE:

```
def fibonacci_recursive(n):  
    if n <= 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)  
  
# Example usage  
fib_number = 10  
print(f"Fibonacci number at position {fib_number} (recursive): {fibonacci_recursive(fib_number)}")
```

```
C: > Users > Shree priya > Downloads > aac5-5.py > is_prime_optimized

28 def fibonacci(n):
29     """
30     This function returns the nth Fibonacci number using recursion.
31     Fibonacci series:
32     0, 1, 1, 2, 3, 5, 8, ...
33     """
34
35     # Base Case 1:
36     # If n is 0, return 0 (first Fibonacci number)
37     if n == 0:
38         return 0
39
40     # Base Case 2:
41     # If n is 1, return 1 (second Fibonacci number)
42     elif n == 1:
43         return 1
44
45     # Recursive Case:
46     # The function calls itself to calculate the previous two Fibonacci numbers
47     else:
48         return fibonacci(n - 1) + fibonacci(n - 2)
49
50 # Example usage
51 num = 6
52 print(f"The {num}th Fibonacci number is: {fibonacci(num)}")

PROBLEMS 2 OUTPUT TERMINAL PORTS

▼ TERMINAL

Optimized: Is 29 prime? True
The 6th Fibonacci number is: 8
Error: The file was not found. Please check the filename.
File processing attempt completed.
Enter username: avara
Enter password: 1286
Invalid username or password
Buddy says Woof!
Buddy is 3 years old.
Enter username: avara
Enter password: 1234
Invalid username or password
Buddy says Woof!
```

Task Description #3 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

CODE:

```
def divide_numbers(a, b):  
    """Divide two numbers with error handling."""  
    try:  
        result = a / b  
    except ZeroDivisionError:  
        return "Error: Cannot divide by zero."  
    except TypeError:  
        return "Error: Both inputs must be numbers."  
    else:  
        return result  
  
# Example usage  
num1 = 10  
num2 = 0  
print(f"Dividing {num1} by {num2}: {divide_numbers(num1, num2)}")  
  
num2 = 2  
print(f"Dividing {num1} by {num2}: {divide_numbers(num1, num2)}")
```

```
> Users > Shree priya > Downloads > aac5-5.py > Dog > bark
55 #Python Code with Meaningful Exception Handling
56 def read_and_process_file(filename):
57     """
58     This function reads a file containing numbers (one per line),
59     converts them to integers, and calculates their sum.
60     """
61     total = 0
62     try:
63         # Try to open the file in read mode
64         file = open(filename, "r")
65         # Read file line by line
66         for line in file:
67             # Remove whitespace and convert to integer
68             number = int(line.strip())
69             total += number
70         file.close()
71         print("Sum of numbers:", total)
72     except FileNotFoundError:
73         # This block executes if the file does not exist
74         print("Error: The file was not found. Please check the file")
75     except ValueError:
76         # This block executes if conversion to integer fails
77         print("Error: The file contains non-numeric data.")
78     except Exception as e:
79         # This block handles any unexpected error
80         print(f"An unexpected error occurred: {e}")
```

PROBLEMS 2 OUTPUT TERMINAL PORTS

▼ **TERMINAL**

```
Optimized: Is 29 prime? True
The 6th Fibonacci number is: 8
Error: The file was not found. Please check the filename.
File processing attempt completed.
Enter username: avara
Enter password: 1286
Invalid username or password
Buddy says Woof!
Buddy is 3 years old.
Enter username: avara
Enter password: 1234
Invalid username or password
Buddy says Woof!
```

Task Description #4 (Security in User Authentication)

Task: Use an AI tool to generate a Python-based login system.

Analyze: Check whether the AI uses secure password handling practices.

Expected Output:

- Identification of security flaws (plain-text passwords, weak validation).

- Revised version using password hashing and input validation.
- Short note on best practices for secure authentication.

CODE:

```
users = {
    "admin": "admin123",
    "user1": "password"
}

username = input("Enter username: ")
password = input("Enter password: ")

if username in users and users[username] == password:
    print("Login successful")
else:
    print("Invalid username or password")

# Example usage
my_dog = Dog("Buddy", 3)
print(my_dog.bark())
print(my_dog.get_age())

import hashlib

users = {
    "admin": hashlib.sha256("admin123".encode()).hexdigest(),
    "user1": hashlib.sha256("securepass".encode()).hexdigest()
}

def login():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()

    if not username or not password:
        print("Error: Username and password cannot be empty.")
        return

    hashed_password = hashlib.sha256(password.encode()).hexdigest()

    if username in users and users[username] == hashed_password:
        print("Login successful")
```


else:

```
    print("Invalid username or password")
```

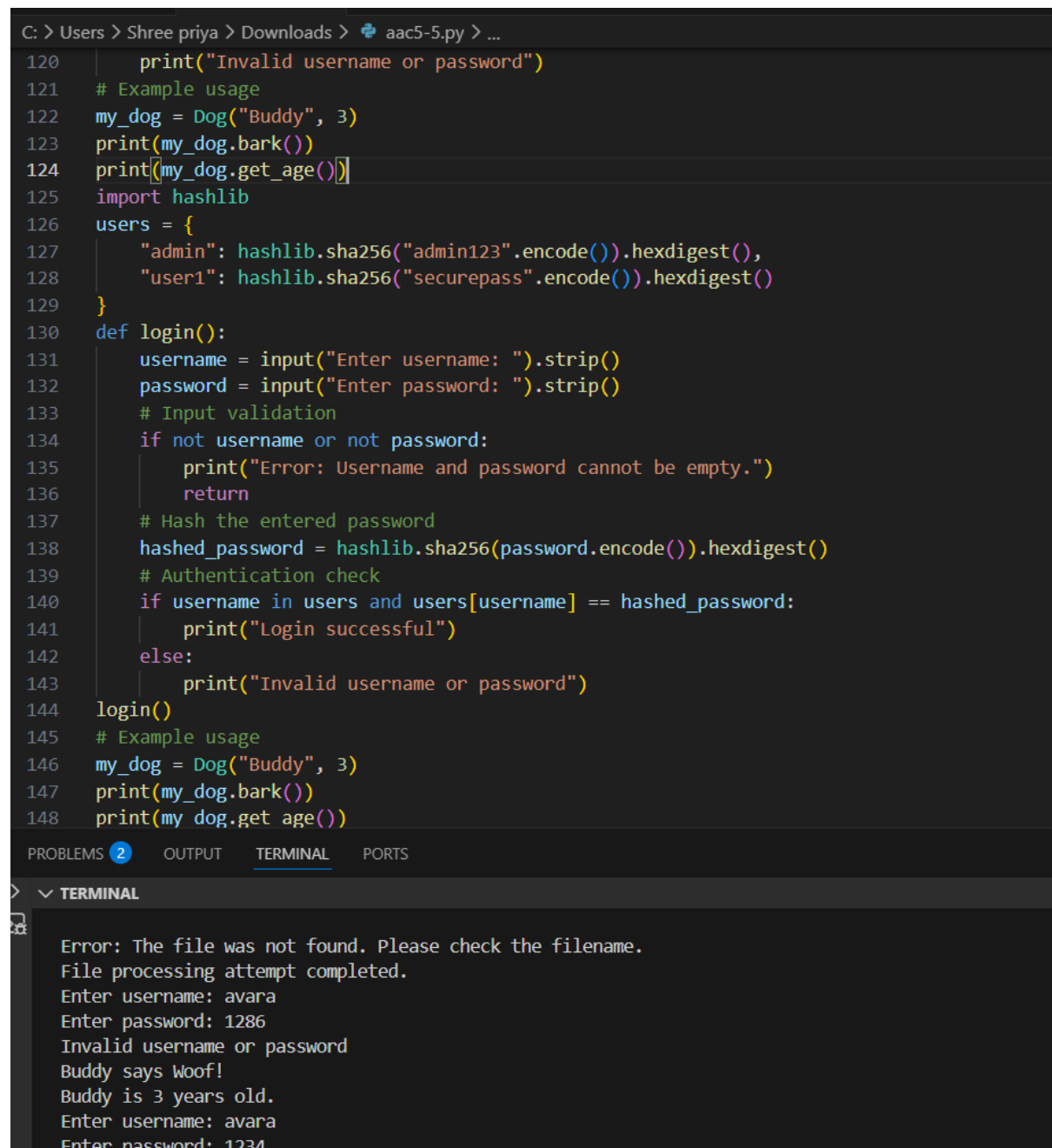
```
login()
```

```
# Example usage
```

```
my_dog = Dog("Buddy", 3)
```

```
print(my_dog.bark())
```

```
print(my_dog.get_age())
```



```
C: > Users > Shree priya > Downloads > aac5-5.py > ...
120     print("Invalid username or password")
121     # Example usage
122     my_dog = Dog("Buddy", 3)
123     print(my_dog.bark())
124     print(my_dog.get_age())
125     import hashlib
126     users = {
127         "admin": hashlib.sha256("admin123".encode()).hexdigest(),
128         "user1": hashlib.sha256("securepass".encode()).hexdigest()
129     }
130     def login():
131         username = input("Enter username: ").strip()
132         password = input("Enter password: ").strip()
133         # Input validation
134         if not username or not password:
135             print("Error: Username and password cannot be empty.")
136             return
137         # Hash the entered password
138         hashed_password = hashlib.sha256(password.encode()).hexdigest()
139         # Authentication check
140         if username in users and users[username] == hashed_password:
141             print("Login successful")
142         else:
143             print("Invalid username or password")
144     login()
145     # Example usage
146     my_dog = Dog("Buddy", 3)
147     print(my_dog.bark())
148     print(my_dog.get_age())
```

PROBLEMS 2 OUTPUT TERMINAL PORTS

▼ TERMINAL

```
Error: The file was not found. Please check the filename.
File processing attempt completed.
Enter username: avara
Enter password: 1286
Invalid username or password
Buddy says Woof!
Buddy is 3 years old.
Enter username: avara
Enter password: 1234
```

Task Description #5 (Privacy in Data Logging)

Task: Use an AI tool to generate a Python script that logs user

activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Expected Output:

- Identified privacy risks in logging.
- Improved version with minimal, anonymized, or masked logging.
- Explanation of privacy-aware logging principles.

CODE:

```
import datetime

def log_activity(username, ip_address):
    timestamp = datetime.datetime.now()
    with open("activity.log", "a") as file:
        file.write(f"{username}, {ip_address}, {timestamp}\n")

import datetime
import hashlib

def anonymize_username(username):
    return hashlib.sha256(username.encode()).hexdigest()[:10]

def mask_ip(ip_address):
    parts = ip_address.split(".")
    parts[-1] = "xxx"
    return ".".join(parts)

def log_activity(username, ip_address):
    timestamp = datetime.datetime.now()
    anon_user = anonymize_username(username)
    masked_ip = mask_ip(ip_address)
    with open("activity.log", "a") as file:
        file.write(f"user={anon_user}, ip={masked_ip}, time={timestamp}\n")

# Example usage

import datetime

def log_activity(username):
```

```

time = datetime.datetime.now()

with open("log.txt", "a") as file:

    file.write(f"user=***, time={time}\n")

log_activity("user1")

```

```

50 #Improved Privacy-Aware Logging
51 import datetime
52 import hashlib
53
54 def anonymize_username(username):
55     # Hash the username to prevent direct identification
56     return hashlib.sha256(username.encode()).hexdigest()[:10]
57
58 def mask_ip(ip_address):
59     # Mask the last part of the IP address
60     parts = ip_address.split(".")
61     parts[-1] = "xxx"
62     return ".".join(parts)
63
64 def log_activity(username, ip_address):
65     timestamp = datetime.datetime.now()
66     anon_user = anonymize_username(username)
67     masked_ip = mask_ip(ip_address)
68     with open("activity.log", "a") as file:
69         file.write(f"user={anon_user}, ip={masked_ip}, time={timestamp}\n")
70
71 # Example usage
72 import datetime
73 def log_activity(username):
74     time = datetime.datetime.now()
75     # Log only necessary information (no IP, no full username)
76     with open("log.txt", "a") as file:
77         file.write(f"user=***, time={time}\n")
78
79 log_activity("user1")

```

PROBLEMS 2 OUTPUT TERMINAL PORTS

✓ **TERMINAL**

```

Invalid username or password
Buddy says Woof!
Buddy is 3 years old.
Enter username: avara
Enter password: 1234
Invalid username or password
Buddy says Woof!
Buddy is 3 years old.
PS C:\Users\Shree priya\Downloads>

```